

分布式内存池驾驭大规模数据处理

从UCX到cudf/pandas/ray/arrow的架构适配

Deep Dive: Distributed Memory Pool Architecture for Data Analytics

CXL Memory Pool

UCX/RDMA

cuDF/pandas

Ray/Arrow

金融量化

核心论点

CXL/RDMA 分布式内存池将内存从「计算附属品」升级为「可池化网络资源」，但现有数据分析工具栈（pandas/cudf/ray/arrow）仍假设内存是计算节点的本地附属物。这种架构错配从 UCX 通信层、Arrow 零拷贝协议、到 DataFrame 内存模型都需要重新设计——不是简单的「换内存」，而是从内存语义、数据搬移、调度单元三个层面进行系统性重构。

① 内存语义重构

内存不再假设本地可寻址
位置无关引用 + 全局元数据

② 数据搬移重构

计算跟随数据 → 数据跟随计算
CXL低延迟使能数据主动推送

③ 调度单元重构

Task/Partition → Memory-tier-aware
分层感知 + 数据局部性优化

目录

01

分布式内存池的技术本质

范式转移 / CXL vs RDMA / 实测数据

02

UCX软件栈：RDMA/CXL通信基石

三层架构 / ucTrace / Dask通信对比

03

主流数据分析工具架构解构

pandas / cuDF / Arrow / Ray / Dask

04

分布式内存池的冲击与适配路径

四层重构：分配器→通信→数据模型
→调度

05

金融量化生态的特殊需求

FinRL / 因子计算 / 回测引擎

06

大规模数据处理架构原理

五条核心设计原理

07

架构设计原则与Roadmap

三阶段演进路径

01 分布式内存池的技术本质

范式转移：内存从计算附属品 → 可池化网络资源

传统：[CPU]←DDR→[DRAM_A] [CPU]←DDR→[DRAM_B] ← 内存孤岛
池化：[CPU]←CXL→[内存池_A]←RDMA→[内存池_B] ← 内存资源池

25-35%

DRAM 内存搁浅
(Aquifer 实测)

10x+

量化因子矩阵
超出单节点 DRAM

CXL-DMSim 硅验证实测 (2411.02282):

CXL-FPGA 延迟 ~2.88x 本地DDR 带宽 45-69%

CXL-ASIC 延迟 ~2.18x 本地DDR 带宽 82-83%

关键：所有应用必须重构数据布局，否则延迟惩罚抵消容量收益

CXL 与 RDMA 的角色分工

维度	CXL	RDMA
访问粒度	字节级 load/store	消息级 RDMA Write/Read
延迟	~200-400ns (CXL.mem)	~1-5μs (InfiniBand)
覆盖范围	Pod级 (<20节点)	集群级 (1000+节点)
透明性	对软件透明 (NUMA感知)	需显式注册内存
Cache一致性	有 (CXL 3.0)	无
典型形态	Type 3 Memory Expander	InfiniBand / RoCE

关键洞察：单一技术无法同时满足低延迟与集群级覆盖。Aquifer (2606.24079) 提出分层：CXL负责热数据(load/store透明)，RDMA负责冷数据(cluster-wide reach)。

CCCL: CXL 替代 RDMA 的 GPU 集合通信 (2602.22457)

1.34x

AllGather

1.84x

Broadcast

1.94x

Gather

1.04x

Scatter

传统 NCCL AllReduce:

GPU_A $\rightarrow$ (RDMA) $\rightarrow$ GPU_B $\rightarrow$ (RDMA) $\rightarrow$ GPU_C

CCCL CXL模式:

GPU_A $\rightarrow$ (CXL.mem写入) $\rightarrow$ 共享内存池 $\rightarrow$ (CXL.mem读取) $\rightarrow$ GPU_B/GPU_C

CXL不再只是「扩展内存」，而是成为集合通信的新传输层

LLM训练 Case Study: 1.11x 加速 + 2.75x 硬件成本节省

(TITAN-II CXL switch + 6x Micron CZ120, vs 200Gbps InfiniBand RDMA)

02 UCX 软件栈：RDMA/CXL 的通信基石

应用层 NCCL / MPI / Dask UCX-Py

UCP (Upper) 协议/接口层 – tag matching, stream, active message

UCT (Transport) InfiniBand RC/UC/UD | RoCE v1/v2 | CUDA IPC | Shared Memory | GPU-NIC Direct

UCS (Services) 内存管理 / 错误处理 / 时间管理 / 系统抽象

IB RC

~1μs | 200-400G

RoCE v2

~2-5μs | 100-400G

CUDA IPC

~100ns | 900GB/s

GPU-NIC

~700ns | 200-400G

ucTrace 与 MPI4Dask: UCX 真实行为揭秘

ucTrace (2602.19084) 关键发现

1. MPI函数与 UCX transport 映射不是一对一
→ 一个 MPI_Allreduce 触发多轮 UCX RDMA
2. NUMA 绑定影响 UCX 性能
→ 跨NUMA GPU-NIC 路径导致 2-3x 下降
3. GPU-aware MPI 必须通过 UCX GPU transport
→ CUDA-aware buffer 选择正确路径是性能关键
4. GROMACS MD 模拟中 UCX 是瓶颈
→ 通信分析需精确到 UCX 层级

MPI4Dask vs UCX-Py (2101.08878)

GPU-aware MVAICH2-GDR 利用 GPUDirect RDMA
GPU内存 → NIC 直达, 完全绕过 CPU/host memory

1字节消息	UCX-Py	基准	~1x
1字节消息	MPI4Dask	6x更低延迟	6x ↑
大消息 (2MB+)	UCX-Py	基准	~1x
大消息 (2MB+)	MPI4Dask	4x更高吞吐	4x ↑
cuPy数组求和	UCX-Py	基准	~1x
cuPy数组求和	MPI4Dask	3.47x	3.47x ↑
cuDF merge	UCX-Py	基准	~1x
cuDF merge	MPI4Dask	3.11x	3.11x ↑

03 主流数据分析工具架构解构

pandas

BlockManager 模型
单进程地址空间
numpy连续内存
△ 冲突：无法引用远端内存

cuDF

GPU 列式模型
RMM 内存池 (per-GPU)
WarpCore Hash Table
△ 冲突：RMM 不管理 CXL

Arrow

列式零拷贝协议
共享内存 IPC
Flight gRPC/TCP
△ 冲突：指针无法跨节点

Ray

不可变对象+引用计数
本地共享内存 Object
Store
全局调度器
△ 冲突：远端内存打破零拷贝

Dask

延迟计算图+工作窃取
中心 Scheduler
TCP/UCX 通信
△ 冲突：Spill-to-disk假设过时

pandas BlockManager 与 cuDF GPU 模型详解

pandas DataFrame → BlockManager

```
DataFrame
├── BlockManager
│   ├── Block_0 (float64) → numpy数组 → 连续内存
│   ├── Block_1 (int64)   → numpy数组 → 连续内存
│   ├── Block_2 (object)  → PyObject* → 堆内存
│   └── ...
```

核心假设:

- 所有数据在进程地址空间内
- copy-on-write 语义
- 字符串列 PyObject 无法序列化

分布式内存池冲突:

- ✗ BlockManager 无法引用 CXL 远端内存
- ✗ copy-on-write 与 CXL 延迟不对称矛盾

cuDF DataFrame → RMM 内存管理

```
cuDF DataFrame
├── Column (GPU buffer)
│   ├── data buffer (GPU VRAM)
│   ├── validity buffer (GPU VRAM)
│   └── child columns
└── RMM 内存池 (per-GPU)
    ├── CUDA memory pool
    └── Sub-allocator
```

内部: WarpCore GPU Hash Table

- 1.6B inserts/s on GV100
- 4.3B retrievals/s
- NVLink 近线性扩展

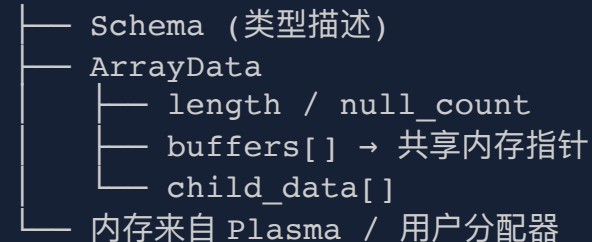
分布式内存池冲突:

- ✗ RMM 只管理本地 GPU VRAM
- ✗ hash bucket 需跨节点重新分布

Arrow 零拷贝协议 与 Ray 分布式调度模型

Apache Arrow 列式零拷贝

Arrow RecordBatch



Arrow Flight 传输 (2204.03032):

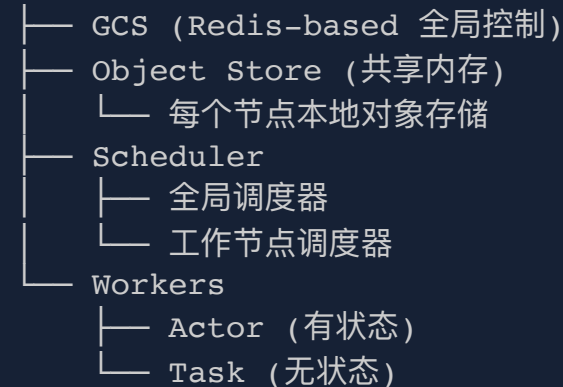
- DoGet: 6000 MB/s
- DoPut: 4800 MB/s
- 带宽利用率: 95% (Mellanox IB)
- 对比 ODBC: 20-30x 提升

冲突:

- ✗ buffers[] 指针假设本地可寻址
- ✗ 序列化开销在 CXL 场景成为瓶颈

Ray 分布式对象模型

Ray Cluster



对象模型: 不可变 + 引用计数

- 跨节点共享 = 序列化/反序列化
- 同节点内 = 共享内存零拷贝

冲突:

- ✗ 不可变对象 vs CXL 共享可变语义
- ✗ 延迟虽低但仍比本地 DRAM 高 2-3x

04 分布式内存池对每层架构的冲击与适配路径

第一层

内存分配器重构

RMM/cuDF + jemalloc/pandas + Arrow allocator

↓

Tiered Memory Allocator

- Local DRAM Tier
- CXL Memory Pool Tier
- NVMe-oF / GDS Tier

第二层

通信层重构

Arrow Flight (gRPC/TCP) Dask (TCP/UCX) cuDF (NCCL)

↓

CXL Shared Memory Transport

- CXL.mem DoGet/DoPut (绕过序列化)

- UCX CXL transport (新)

第三层

数据模型重构

BlockManager / Column / ArrayData

↓

分布式 DataFrame 模型

- 位置无关的 Chunk 引用
- 全局元数据服务

第四层

调度器重构

Task Scheduler / Ray Scheduler / DAGScheduler

↓

CXL-aware Scheduler

- 热页预装 CXL
- 冷页按需 demand-page
- 零页压缩传输

各工具适配路径汇总

工具	当前假设	分布式内存池冲突	适配路径
pandas	单进程地址空间	BlockManager 无法引用远端	Modin + CXL-aware allocator
cuDF	GPU本地VRAM	RMM 不管理 CXL	RMM 扩展 CXL tier + UCX CXL
Arrow	共享内存 IPC	指针无法跨节点	Arrow Flight CXL transport
Ray	不可变对象+本地共享内存	远端内存打破零拷贝	Ray Object Store CXL扩展
Dask	Task图+工作窃取	Spill-to-disk假设过时	CXL-aware task placement
NCCL	RDMA/PCIe	CXL共享内存替代RDMA	CCCL 式 CXL 集合通信

05 金融量化生态的特殊需求与架构响应

01

Tick数据预处理 全市场Level-2清洗

单市场日增量50-200GB

瓶颈: pandas read_csv内存溢出

02

因子矩阵计算 Alpha101/103/WorldQuant

$3000 \times 2000 \times 500 = 3\text{B floats}$

瓶颈: 单节点内存不足

03

回测引擎 vectorbt / backtrader

全市场10年分钟线 ~50GB

瓶颈: DataFrame 反复 copy

04

组合优化 Mean-Variance / Risk Parity

协方差矩阵 3000×3000

瓶颈: 中间结果内存

05

机器学习训练 LightGBM / LSTM / FinRL

特征矩阵可达 100GB+

瓶颈: GPU显存+I/O

FinRL-Podracr: GPU 云大规模 RL 训练 (2111.05188)

FinRL-Podracr 架构

Generational Evolution + Ensemble

- 代际进化机制
- 集成策略提升收益

Multi-level Mapping

- Level 1: 策略 → GPU集群映射
- Level 2: 环境 → GPU节点映射
- Level 3: 数据 → GPU内存映射

GPU Cloud (NVIDIA DGX SuperPOD)

- 80x A100 GPUs
- NASDAQ-100 10年分钟数据
- 10分钟完成训练

12-35%

年化收益提升

0.1-0.6

Sharpe ratio 提升

3-7x

训练时间加速

10min

80 A100 训练耗时

CXL 内存池启示: FinRL-Podracr 的 multi-level mapping 中, 「数据→GPU内存映射」在 CXL 时代变为「数据→CXL池↔GPU内存映射」。当前 10 分钟训练中 30-40% 用于数据加载, CXL 内存池可大幅削减。

因子计算与回测引擎的 CXL 架构重构

因子计算：向量化 → 分布式

当前模式 (内存瓶颈):

```
close = pd.read_parquet('close_matrix.parquet') # 内存峰值!
temp1 = close.rolling(5).apply(np.max)          # 4.8GB
(copy)
temp2 = temp1.rank(axis=1)                       # 7.2GB
(copy)
alpha002 = -1 * close.rolling(5).corr(volume)   # 9.6GB
(copy)
```

CXL 池化模式:

```
close = CXLDDataFrame.from_parquet(
    'close_matrix.parquet',
    pool='cxl_pool_A',
    location_policy='hot_columns')
# 中间结果直接写入 CXL 池, 不占用本地 DRAM
alpha001 = close.rolling(5).max().rank(
    axis=1, result_pool='cxl_pool_A')
```

回测引擎：内存模型重构

传统回测引擎假设:

- OHLCV 全量驻留本地内存
- 反复切片/滚动窗口
- 信号矩阵同大小

CXL 内存池重构:

- OHLCV 驻留 CXL 池
- 预取策略: 热数据 (当前窗口) → 本地
- 信号矩阵分布式存储在多个 CXL 池
- 策略状态 (小) 保留在本地 DRAM

核心变更:

「数据加载到计算」

↓

「计算跟随数据位置」

06 大规模数据处理的架构原理

原理一·数据局部性分层

L1 GPU VRAM → L2 Local DRAM → L3 CXL Pool → L4 RDMA Pool → L5 NVMe-oF

热数据→L1/L2 | 温数据→L3 | 冷数据→L4 | 归档→L5

原理二·计算-存储分离极致化

传统：计算→数据位置（高网络开销）

CXL：数据→CXL池（低延迟共享）

多节点同时 load/store 访问同一内存区域

原理三·通信-计算重叠

UCX 异步 API 使能：

[Compute_A] [CXL.mem write] [Compute_B] [CXL.mem read]
└─── 重叠 ───┘ └─── 重叠 ───┘

原理四·序列化开销消除

Arrow Flight: Serialize → TCP → Deserialize (80%时间)

CXL Flight: CXL.mem write → Pool → CXL.mem read (接近零)

原理五·容错模型转变

传统：数据复制到3个Worker (3x存储)

CXL：数据写入CXL池 (1x存储)

天然「复制」= 多节点同时读取

新内存层次：从 GPU VRAM 到 NVMe-oF

L1: GPU VRAM

延迟: ~100ns

带宽: 2-3TB/s

容量: 80-800GB

L2: Local DRAM

延迟: ~100ns

带宽: 100GB/s

容量: 0.5-2TB

设计原则: 热数据(当前计算窗口)→L1/L2 | 温数据(近期历史)→L3 CXL | 冷数据(全量历史)→L4 RDMA | 归档→L5 NVMe-oF

07 架构设计原则与 Roadmap

位置透明

应用不假设数据物理位置
全局元数据服务记录 Chunk
位置

分层感知

调度器感知 L1-L5 内存层次
数据局部性优化

零拷贝优先

同节点共享内存
远端 CXL 直访

异步解耦

计算与数据搬运异步执行
UCX 异步 API 使能

弹性伸缩

CXL 池容量独立于计算扩展
内存池即服务 (MPaaS)

Phase 1 2024-2025

CXL Type 3 Memory Expander 硬件就绪

- RMM/cuDF: 扩展内存分配器支持 CXL tier
- Arrow Flight: 实验性 CXL transport
- 金融应用: 因子矩阵全量驻留 CXL 池

Phase 2 2025-2026

软件 cache 一致性

- UCX: 官方 CXL transport
- NCCL: CCCL 式 CXL 集合通信
- Ray/Dask: CXL-aware 调度器
- 金融应用: 分布式回测引擎

Phase 3 2026-2028

- pandas/cudf: 原生分布式 DataFrame
- Arrow: 全局共享缓冲区
- 金融应用: 实时因子+回测+优化一体化
- 新范式: 内存池即服务 (MPaaS)

关键技术风险



CXL 2.0 无硬件 cache coherence

需要软件协议保证一致性
(Aquifer ownership-based coherence)
编程复杂度显著增加



CXL 交换机拓扑限制

当前 CXL switch 支持有限节点数
大规模部署需要多级交换
成本与复杂度上升



安全隔离

共享内存池需要新的安全模型
参考 NeVerMore 对 RDMA 的安全分析
多租户隔离是核心挑战



软件生态碎片化

UCX/Arrow/Ray/Dask 各自为战
缺乏统一的 CXL 编程模型
标准化进程缓慢

Reference Map

CXL 内存池

2411.02282 CXL-DMSim: 硅验证全系统 CXL 解耦内存模拟器 (3.4% 误差)

2606.24079 Aquifer: 分层 CXL+RDMA 内存池 (2.2x MicroVM 启动加速)

2503.17864 CXL Tiered Memory: 架构与系统影响

2602.22457 CCCL: CXL 替代 RDMA 的 GPU 集合通信 (1.34-1.94x 提升)

2211.02682 CXL HPC Evaluation: CXL 内存池在 HPC 中的潜力

2404.03551 CXL Hyperscale: 超大规模 CXL 部署 TCO 优化

UCX / RDMA 通信

2602.19084 ucTrace: 首个 UCX 层级细粒度 profiling 工具

2507.04786 Demystifying NCCL: NCCL 内部协议/算法全面分析

2511.15076 GIN for NCCL: GPU-Initiated Networking (DOCA GPUNetIO)

2101.08878 MPI4Dask: GPU-aware MPI 比 UCX 快 3.47x

2608.01449 NIXT: NCCL Inspector 数据导出与分析

2502.08982 Outback: 解耦内存高效 KV 索引 (1.06-5.03x 吞吐)

Reference Map (cont)

数据分析工具

- 2204.03032 Arrow Flight Benchmark: 线速传输 6000MB/s DoGet
- 2106.13020 Arrow-Spark Zero-Cost: Arrow 零成本数据互操作层
- 2010.11105 Dask Runtime vs Scheduler: 运行时开销是主要瓶颈
- 2001.00888 Towards Scalable Dataframe: 可扩展 DataFrame 系统愿景
- 2009.07914 WarpCore: GPU 哈希表库 (1.6B inserts/s)

金融量化

- 2111.09395 FinRL: 首个开源深度 RL 量化交易框架
- 2111.05188 FinRL-Podracers: 80 A100 10分钟训练 (12-35% 收益提升)

解耦内存系统

- 2306.04014 Disaggregated Memory HPC: 13个负载的解耦内存评估
- 2404.03155 TEGRA: 解耦计算上的万亿级图处理
- 2202.08080 NeVerMore: RDMA/NVMe-oF 安全漏洞分析

结语

分布式内存池不是「更快的SSD」或「更大的DRAM」，
而是一种新的计算范式——
内存从计算附属品变为可池化、可共享、可独立扩展的网络资源。

对于金融量化生态而言：
因子矩阵可以全量驻留内存池而无需分批
回测引擎可以实时访问全量历史数据
RL 训练可以在分钟级完成

但实现这一愿景需要从 UCX 通信层、Arrow 协议层、
DataFrame 内存模型到调度器的全栈重构。

CCCL 和 Aquifer 已经指明了方向，
下一步是软件生态的系统性适配

核心结论

从「内存即附属」到「内存即资源」的范式转移，要求数据分析工具栈在 内存分配器 → 通信层 → 数据模型 → 调度器 四个层面进行系统性重构。金融量化场景（因子计算/回测/RL训练）将是这一变革的最大受益者。

分布式内存池 · 软件架构适配

从 UCX 到 cudf/pandas/ray/arrow 的全栈重构

Hermes Research | 2026

backyes.github.io

Key References

CCCL (2602.22457) | Aquifer (2606.24079) | CXL-DMSim (2411.02282)

ucTrace (2602.19084) | Demystifying NCCL (2507.04786) | FinRL-Podracar (2111.05188)